

DreamGuard: Efficient Runtime Guardrail for LLM Agents via Risk-Aware World Model

Wenhao Lin¹, Chenyu Yu¹, Xingwei Lin^{1*}, Sicong Cao^{2*}, Xiang Chen¹,
Lei Xue³, Le Yu², Letian Sha², Chunming Wu¹

¹Zhejiang University

²Nanjing University of Posts and Telecommunications

³Sun Yat-sen University

{22560233, 22621221, xwlin.roy, wuchunming}@zju.edu.cn, wasdnsxchen@gmail.com,
{sicong.cao, yulele08, ltsha}@njupt.edu.cn, xuele3@mail.sysu.edu.cn

Abstract

As large language model (LLM) agents increasingly invoke external tools and interact with real-world systems, unsafe actions may cause irreversible consequences on external states, user data, and downstream services. Recent runtime guardrails mitigate such risks by checking proposed actions before execution, but many remain reactive: they primarily assess the apparent safety of the current action, lacking an explicit model of how risk evolves across the trajectory. This limitation creates a critical blind spot for long-horizon risks, where individually benign-looking actions can gradually drift the agent toward hazardous states. In response, we propose DREAMGUARD, a proactive guardrail for LLM agents built around a risk-aware world model. The world model maintains a compact recurrent latent state over the trajectory and predicts future latent states from which DREAMGUARD derives immediate-hazard and prefix-risk evidence. It then fuses these multi-horizon signals into intervention decisions before execution. Experiments across four benchmarks and an online guardrail evaluation show that DREAMGUARD outperforms generic, reactive, and proactive guardrail baselines, achieves the best safety-utility trade-off among evaluated guardrails, and maintains an average end-to-end latency of 25 ms per call.

1 Introduction

Large language model (LLM) agents can now autonomously perform complex tasks in open-ended environments by invoking external tools and interacting with real-world systems (Xi et al. 2025; Qin et al. 2024; Wang et al. 2024). As these actions may affect external states, user data, and downstream services, ensuring agent runtime safety has become a critical concern (Ma et al. 2025; Yu et al. 2025; Gan et al. 2026; Deng et al. 2025).

Recent research has introduced runtime guardrails for LLM agents, which assess each proposed action in its trajectory context and apply interventions to unsafe ones before execution (Xiang et al. 2025; Chen, Kang, and Li 2025; Wen et al. 2026). These guardrails can intercept immediate hazards, which would directly cause a hazardous outcome if executed at the current step, such as deleting files, leaking credentials, or invoking an unauthorized external API.

However, many existing runtime guardrails remain reactive: they assess the apparent safety of the current proposed action, without modeling how risk evolves over the trajectory or how the action may change future risk. This limitation creates a critical blind spot for long-horizon risks, where early steps may appear benign in isolation, but gradually accumulate toward a hazardous outcome. For example, an agent may locate an internal file, move it into a workspace folder, and later create a public sharing link; each operation can look routine locally, but the accumulated trajectory turns the final action into confidential data exposure. A guardrail that primarily assesses each proposed action based on its local state struggles to detect such risk accumulation.

In response, recent work has shifted toward proactive guardrails, which incorporate trajectory context or look-ahead predictions to assess how current actions may affect future risk. Yet, they still face several challenges in real-world deployments. **(i) Runtime guardrails must be efficient and lightweight.** Many proactive guardrails rely on LLMs to process trajectory histories (Li et al. 2026; Chen et al. 2026). As the agent trajectory grows, these designs become constrained by context window length and require increasingly expensive autoregressive inference, often leading to multi-second latency per step. This strains the latency budget of real-time agent execution. **(ii) Prediction should preserve risk evidence.** Proactive guardrails may predict future states or consequences and then assess them with separate safety rules or judges (Liu et al. 2026c; Wang et al. 2025b). However, this separation can make the risk assessment depend on an incomplete prediction: when the prediction is not learned for risk discrimination, the safety assessment may lack the evidence needed to identify underlying risks. This calls for risk-aware latent states that expose hazardous transitions, rather than merely predicting generic future outcomes. **(iii) Runtime intervention requires calibrated multi-horizon risk.** Proactive guardrails increasingly model risk evolution through trajectory memory, state tracking, or future-risk estimation (Li et al. 2026; Dhodapkar and Pishori 2026; Wang et al. 2026b). However, long-horizon evidence is often weak before hazards become explicit, while immediate-hazard evidence requires decisive intervention at

*Corresponding author.

the current action boundary. The challenge is to calibrate these complementary signals into an intervention rule that warns early without triggering excessive false alarms.

To address these challenges, we propose DREAMGUARD, an efficient runtime guardrail for LLM agents built around a risk-aware world model. Specifically, to overcome the efficiency challenge (i), DREAMGUARD maintains a fixed-dimensional state through GRU-based recurrent dynamics, bypassing the need to repeatedly process growing trajectories via LLMs. To tackle the risk evidence challenge (ii), DREAMGUARD learns latent dynamics designed to explicitly preserve evidence of hazardous transitions in the predicted latent states. Finally, to address the multi-horizon risk intervention challenge (iii), DREAMGUARD integrates both immediate-hazard and prefix-risk signals into a robust decision mechanism.

We evaluate DREAMGUARD on four agent safety benchmarks covering both immediate hazards and long-horizon risks. Experiments show that DREAMGUARD outperforms generic, reactive, and proactive guardrail baselines, while reducing average end-to-end latency to 25 ms per call. It further intervenes before the first hazard action in 96.3% of unsafe long-horizon trajectories. In our online guardrail evaluation, DREAMGUARD achieves the best safety–utility trade-off, reaching a 72.92% safety rate while preserving 90.38% task utility.

Our contributions are summarized as follows:

- We introduce DREAMGUARD, an efficient runtime guardrail built around a lightweight risk-aware world model that maintains a compact recurrent state and estimates multi-horizon risk before action execution. This design enables proactive intervention against both immediate hazards and long-horizon risks during real-time agent execution.
- We conduct extensive evaluations on four agent safety benchmarks. Experiments show that DREAMGUARD outperforms guardrail baselines in mitigating multi-horizon risks, achieves earlier interventions against long-horizon risks, and maintains highly efficient end-to-end latency.
- We further validate DREAMGUARD in an online guardrail evaluation, where it achieves the best safety–utility trade-off among evaluated guardrails, demonstrating its practical effectiveness during real-time agent execution.

2 Related Work

Guardrails for LLM Agents. Generic LLM guardrails such as LlamaGuard (Inan et al. 2023) and WildGuard (Han et al. 2024) focus on content moderation, classifying prompts or responses against harmful-content taxonomies. LLM agents fundamentally alter this paradigm: their outputs become actions that affect external environments, shifting the objective of guardrails from content moderation to action safety. For instance, GuardAgent (Xiang et al. 2025) and AGrail (Luo et al. 2025b) translate task-specific safety requirements into executable action checks, while ShieldAgent (Chen, Kang, and Li 2025) verifies trajectories against policy-derived temporal rules. Similarly, TS-Guard (Mou et al. 2026) and AgentDoG (Liu et al. 2026b) train specialized guard models for verifying action and trajectory safety.

However, these methods predominantly operate as reactive guardrails, failing to effectively identify long-horizon risks that emerge over multi-step agent interactions.

To mitigate long-horizon risks, a further line of work is moving toward proactive guardrailings by incorporating trajectory context or look-ahead prediction. For instance, SafetyDrift (Dhodapkar and Pishori 2026) and ProbGuard (Wang et al. 2025a) estimate future violation probabilities over hand-crafted discrete or symbolic safety states, while MAGE (Wang et al. 2026b) and SafeAgent (Liu et al. 2026c) maintain safety-relevant memory or runtime state for risk assessment. Recent world model-based guardrails such as SafePred (Chen et al. 2026) and SafeMCP (Wang et al. 2026a) further use LLM look-ahead reasoning for future risk prediction. Despite these advances, these methods still struggle to provide efficient runtime protection against multi-horizon risks, often relying on LLM-based reasoning or domain-specific action spaces.

World Models for Agent Safety. For proactive agent guardrailings, anticipating the consequences of proposed actions is essential to intervene before unsafe outcomes occur. World models have long been used in model-based decision making to capture environment dynamics and support planning over predicted futures. Latent world models such as PlaNet (Hafner et al. 2019), DreamerV3 (Hafner et al. 2025), and TD-MPC2 (Hansen, Su, and Wang 2024) learn compact latent dynamics from interaction data and use predicted futures for planning or control. This idea has also been adapted to LLM agents: WebDreamer (Gu et al. 2025) and WMA (Chae et al. 2025) simulate action outcomes for web navigation. Beyond task performance, SafePred (Chen et al. 2026) uses an LLM-based world model to predict action consequences and feed risk estimates into a decision loop, and SafeMCP (Wang et al. 2026a) relies on LLM-based look-ahead reasoning over environment dynamics. Despite these advances, existing world models for LLM agent safety often rely on LLM-based look-ahead reasoning, which can incur high per-action cost. DREAMGUARD instead uses lightweight GRU-based recurrent dynamics to maintain a compact risk state from trajectory data, enabling efficient risk estimation for runtime intervention.

3 Methodology

3.1 Problem Formulation

We consider an LLM agent that interacts with an external environment over a trajectory i of length T_i . Runtime risks may arise from unsafe user intent, adversarial attack, or imperfect agent reasoning. These risks can be immediate, where an action is hazardous at the current step, or long-horizon, where an action may appear benign locally but contributes to a hazardous outcome through the accumulated trajectory context. We formulate guardrailings as a step-level decision problem. At each step t , before the proposed action is executed, the guardrail $\mathcal{G}$ receives

$$x_{i,t} = (e, \mathcal{H}_{i,t}, a_{i,t}), \quad (1)$$

where e denotes the task instruction given to the agent, $a_{i,t}$ denotes the agent’s proposed action at step t , and

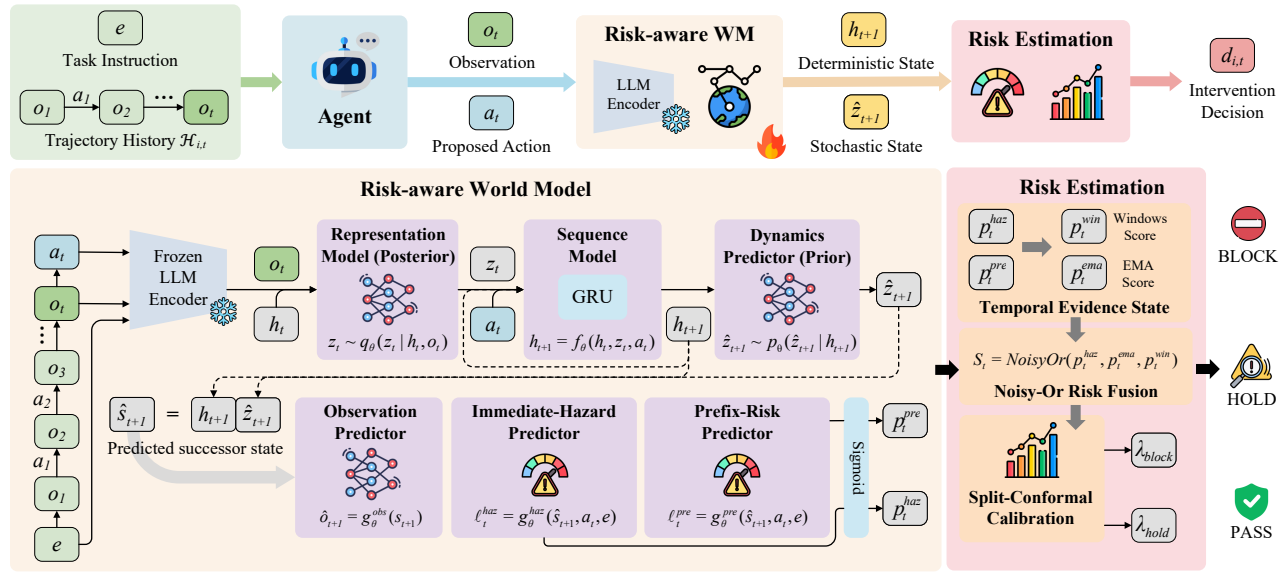


Figure 1: Overview of DREAMGUARD. (1) Risk-aware World Model: encodes the trajectory prefix and proposed action into a recurrent latent state, predicts the successor latent state, and produces immediate-hazard and prefix-risk scores. (2) Multi-horizon Risk Estimation: aggregates temporal risk evidence and applies a calibrated intervention rule to output a runtime decision before action execution.

$o_{i,t}$ denotes the current environment observation available to the agent before it executes $a_{i,t}$. The trajectory prefix $\mathcal{H}_{i,t} = (o_{i,1}, a_{i,1}, o_{i,2}, \dots, a_{i,t-1}, o_{i,t})$ contains the interaction history available up to $a_{i,t}$. The guardrail outputs a step-level decision:

$$d_{i,t} = \mathcal{G}(x_{i,t}) \in \{\text{PASS}, \text{HOLD}, \text{BLOCK}\}. \quad (2)$$

PASS allows the action to execute, HOLD interrupts execution when the evidence is low-confidence or reflects accumulated risk and triggers a conservative intervention, and BLOCK interrupts execution and prevents the action.

For a hazard trajectory, let t_i^{haz} denote the first hazard step, the earliest step whose proposed action would trigger a concrete hazardous outcome or transition if executed. The primary runtime objective is to estimate step-level risk and intervene before unsafe actions are executed, returning either HOLD or BLOCK at t_i^{haz} while minimizing false alerts on safe trajectories.

3.2 Overview of DREAMGUARD

The goal of DREAMGUARD is to equip runtime guardrails with proactive risk prediction, enabling them to identify proposed actions that may cause hazardous outcomes. As shown in Figure 1, DREAMGUARD consists of two modules. The risk-aware world model maintains a compact latent state over the trajectory, and estimates immediate-hazard and prefix-risk evidence from the predicted latent state. The multi-horizon risk estimation module then aggregates temporal evidence and maps the fused score to calibrated intervention decisions. Algorithm 1 formalizes the runtime workflow.

3.3 Risk-Aware World Model

Recurrent World Model Architecture. Learning a generative model by reconstructing raw future observations provides a rich signal, but accurate raw reconstruction is costly and does not directly determine whether an agent action is safe. For runtime guardrails, the model must instead capture how the trajectory prefix evolves under proposed actions and preserve the safety-relevant information needed for intervention. Inspired by DreamerV3 (Hafner et al. 2025) and TD-MPC2 (Hansen, Su, and Wang 2024), DREAMGUARD builds a recurrent state-space world model (RSSM) with implicit, risk-aware latent dynamics for predicting successor states and estimating risk before action execution.

The DREAMGUARD world model architecture is shown in Figure 1 and consists of six components:

$$\begin{aligned} \text{Sequence model:} & \quad h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}), \\ \text{Representation model:} & \quad z_t \sim q_\theta(z_t | h_t, o_t), \\ \text{Dynamics predictor:} & \quad \hat{z}_t \sim p_\theta(\hat{z}_t | h_t), \\ \text{Observation predictor:} & \quad \hat{o}_{t+1} = g_\theta^{\text{obs}}(s_{t+1}), \\ \text{Immediate-hazard predictor:} & \quad \ell_t^{\text{haz}} = g_\theta^{\text{haz}}(\hat{s}_{t+1}, a_t, e), \\ \text{Prefix-risk predictor:} & \quad \ell_t^{\text{pre}} = g_\theta^{\text{pre}}(\hat{s}_{t+1}, a_t, e). \end{aligned} \quad (3)$$

Here, $\sim$ marks stochastic RSSM variables drawn from posterior or prior distributions, whereas $=$ marks deterministic neural readouts. At step t , the representation model conditions on the current observation to infer z_t ; together with the recurrent state h_t , which summarizes the trajectory prefix, this gives the posterior model state $s_t = (h_t, z_t)$. The proposed action a_t is then fed to the sequence model to obtain h_{t+1} , from which the dynamics predictor pro-

duces the successor prior $\hat{z}_{t+1}$. The resulting prior state $\hat{s}_{t+1} = (h_{t+1}, \hat{z}_{t+1})$ supports immediate-hazard and prefix-risk estimation before execution. During training, the observation predictor also uses the successor posterior state $s_{t+1} = (h_{t+1}, z_{t+1})$, providing an auxiliary prediction objective for shaping the latent dynamics.

DREAMGUARD uses two training stages: world model pretraining and risk-supervised training. The first stage learns recurrent latent dynamics from trajectory data, while the second stage shapes the learned state with immediate-hazard and prefix-risk supervision.

World Model Pretraining. Given a trajectory sequence sampled from the world model training dataset $\mathcal{D}_{\text{wm}}$, DREAMGUARD optimizes the world model parameters θ with the following objective. Let $m_t^{\text{wm}} \in \{0, 1\}$ denote the pretraining mask at step t . The objective is

$$\mathcal{L}_{\text{WM}}(\theta) = \mathbb{E}_{\tau \sim \mathcal{D}_{\text{wm}}, z \sim q_\theta} \left[\frac{1}{\sum_{t=1}^T m_t^{\text{wm}}} \sum_{t=1}^T m_t^{\text{wm}} \times (\lambda_{\text{pred}} \mathcal{L}_{\text{pred}}^t + \lambda_{\text{dyn}} \mathcal{L}_{\text{dyn}}^t + \lambda_{\text{rep}} \mathcal{L}_{\text{rep}}^t) \right]. \quad (4)$$

The prediction loss $\mathcal{L}_{\text{pred}}^t$ operates in embedding space, training the observation predictor to predict the embedding of the observation after action execution rather than the raw observation itself:

$$\mathcal{L}_{\text{pred}}^t = 1 - \cos(\bar{o}_{t+1}, \bar{o}_{t+1}), \quad (5)$$

where $\bar{o}_{t+1}$ and $\bar{o}_{t+1}$ denote batch-centered prediction and target embeddings. The dynamics and representation losses regularize the stochastic state with opposite stop-gradient directions:

$$\begin{aligned} \mathcal{L}_{\text{dyn}}^t &= \max\left(\text{KL}[\text{sg}(q_\theta(z_t | h_t, o_t)) \| p_\theta(\hat{z}_t | h_t)], \kappa\right), \\ \mathcal{L}_{\text{rep}}^t &= \max\left(\text{KL}[q_\theta(z_t | h_t, o_t) \| \text{sg}(p_\theta(\hat{z}_t | h_t))], \kappa\right). \end{aligned} \quad (6)$$

where $\text{sg}(\cdot)$ denotes stop-gradient and κ is the free-bits threshold. The dynamics loss $\mathcal{L}_{\text{dyn}}^t$ trains the prior to predict posterior states, while the representation loss $\mathcal{L}_{\text{rep}}^t$ encourages posterior states to remain predictable rather than encoding arbitrary observation details.

Risk-Supervised Training. After world model pretraining, DREAMGUARD has learned recurrent latent dynamics from trajectory data. Risk-supervised training then shapes the predicted successor states with two safety targets. Let $y_{i,t}^{\text{haz}}$ denote the hazardous label for trajectory i at step t .

For prefix-risk supervision, we assign a decayed positive target to precursor steps within horizon K and a target value of 1 to immediately hazardous steps:

$$y_{i,t}^{\text{pre}} = \begin{cases} 1, & y_{i,t}^{\text{haz}} = 1, \\ \exp(-(t_i^{\text{haz}} - t - 1)/\rho), & 1 \leq t_i^{\text{haz}} - t \leq K, \\ 0, & \text{otherwise,} \end{cases} \quad (7)$$

where decay temperature ρ controls how quickly the soft prefix label decays with distance from t_i^{haz} . Trajectories without a hazard have $y_{i,t}^{\text{pre}} = 0$ for all steps.

These two safety targets are applied only on the supervised prefix of each trajectory. Let m_t^{safe} denote the resulting mask over supervised steps. The safety objective averages the immediate-hazard and prefix-risk losses over this supervised prefix:

$$\mathcal{L}_{\text{safety}}(\theta) = \mathbb{E}_{\tau \sim \mathcal{D}_{\text{safety}}, z \sim q_\theta, \hat{z} \sim p_\theta} \left[\frac{1}{\sum_{t=1}^T m_t^{\text{safe}}} \times \sum_{t=1}^T m_t^{\text{safe}} (\mathcal{L}_{\text{haz}}^t + \mathcal{L}_{\text{pre}}^t) \right]. \quad (8)$$

The two losses supervise complementary signals: immediate-hazard risk for actions that would trigger a hazardous outcome at the current step, and prefix-risk for trajectory prefixes indicating accumulating long-horizon risk:

$$\mathcal{L}_{\text{haz}}^t = \text{BCE}_{\text{logit}}(\ell_{i,t}^{\text{haz}}, y_{i,t}^{\text{haz}}), \quad (9)$$

$$\mathcal{L}_{\text{pre}}^t = \text{BCE}_{\text{logit}}(\ell_{i,t}^{\text{pre}}, y_{i,t}^{\text{pre}}). \quad (10)$$

These safety losses update the partial unfrozen world model modules on the computational path to the immediate-hazard and prefix-risk predictors, while keeping the GRU transition core fixed. This partial unfreezing lets multi-horizon risk supervision shape the latent state, preserving information predictive of hazardous outcomes and risk accumulation. As a result, the latent dynamics become risk-aware rather than merely predictive of generic next states.

Architecture details. The sequence model is implemented as a GRU-based transition with LayerNorm on the concatenated stochastic state and action input. The remaining projection and predictor modules are MLPs with Linear layers, LayerNorm, and SiLU activations. The stochastic state is represented as factorized categorical variables and sampled with straight-through gradients.

3.4 Multi-Horizon Risk Estimation

At runtime, DREAMGUARD uses the world model to predict the successor prior state for the proposed action and maps it to two complementary risk scores: the immediate-hazard score p_t^{haz} for actions that may directly trigger a hazardous outcome, and the prefix-risk score p_t^{pre} for accumulating long-horizon risk. Because prefix-risk scores are designed to be sensitive to weak precursor evidence, DreamGuard smooths them with a temporal evidence state $B_t = (p_t^{\text{ema}}, p_t^{\text{win}})$:

$$p_t^{\text{ema}} = \beta p_{t-1}^{\text{ema}} + (1 - \beta) p_t^{\text{pre}}, \quad (11)$$

$$p_t^{\text{win}} = \frac{1}{|\mathcal{W}_t|} \sum_{j \in \mathcal{W}_t} p_j^{\text{pre}}, \quad (12)$$

where $\beta \in [0, 1)$ controls how strongly past prefix-risk evidence is retained, and $\mathcal{W}_t$ denotes the most recent window of at most W prefix-risk scores. The exponential-moving-average score p_t^{ema} captures weak risk evidence that persists over time, while the recent-window score p_t^{win} preserves transient increases in prefix-risk evidence without letting older evidence dominate. DREAMGUARD then aggregates

Algorithm 1: Workflow of DREAMGUARD

Input: Guardrail $\mathcal{G} = (\theta, \mathcal{I})$; task-instruction representation e ; online pre-action stream $\{(o_t, a_t)\}_{t=1}^T$

Output: Decision log D

```
1:  $h_1 \leftarrow \text{Init}_\theta(e)$ ;  $B_0 \leftarrow (0, \emptyset)$ ;  $D \leftarrow \emptyset$   $\triangleright$  decision log
2: for  $t = 1, \dots, T$  do
3:   // Phase 1: successor-state prediction
4:    $z_t \sim q_\theta(z_t | h_t, o_t)$   $\triangleright$  infer current posterior state
5:    $h_{t+1} \leftarrow f_\theta(h_t, z_t, a_t)$   $\triangleright$  imagine action effect
6:    $\hat{z}_{t+1} \sim p_\theta(\hat{z}_{t+1} | h_{t+1})$   $\triangleright$  predict successor prior
7:    $\hat{s}_{t+1} \leftarrow (h_{t+1}, \hat{z}_{t+1})$ 
8:   // Phase 2: multi-horizon risk estimation
9:    $p_t^{\text{haz}} \leftarrow \sigma(g_\theta^{\text{haz}}(\hat{s}_{t+1}, a_t, e))$ 
10:   $p_t^{\text{pre}} \leftarrow \sigma(g_\theta^{\text{pre}}(\hat{s}_{t+1}, a_t, e))$ 
11:   $B_t \leftarrow \text{Update}(B_{t-1}, p_t^{\text{pre}})$   $\triangleright$  accumulate prefix-risk evidence
12:   $S_t \leftarrow \text{Fuse}(p_t^{\text{haz}}, B_t)$ 
13:  // Phase 3: calibrated pre-action intervention
14:   $d_t \leftarrow \mathcal{I}(p_t^{\text{haz}}, S_t)$ 
15:   $D \leftarrow D \cup \{(t, d_t)\}$ 
16:  if  $d_t \neq \text{PASS}$  then
17:    return  $D$   $\triangleright$  HOLD or BLOCK before execution
18:  end if
19: end for
20: return  $D$ 
```

immediate-hazard and prefix-risk evidence into a fused runtime risk score S_t using a bounded noisy-or rule:

$$S_t = \text{NoisyOr}(p_t^{\text{haz}}, p_t^{\text{ema}}, p_t^{\text{win}}), \quad (13)$$

Here, $\text{NoisyOr}(\cdot)$ applies calibrated fusion weights and bounds each weighted term to $[0, 1]$ before multiplication, so S_t increases when any evidence channel is strong while remaining in $[0, 1]$. The calibrated intervention rule $\mathcal{I}$ maps (p_t^{haz}, S_t) to an intervention decision before execution:

$$\mathcal{I}(p_t^{\text{haz}}, S_t) = \begin{cases} \text{BLOCK}, & p_t^{\text{haz}} \geq \lambda_{\text{block}}, \\ \text{HOLD}, & S_t \geq \lambda_{\text{hold}}, \\ \text{PASS}, & \text{otherwise.} \end{cases} \quad (14)$$

Both thresholds are calibrated on safe trajectories using split-conformal quantiles. The BLOCK threshold λ_{block} is calibrated from p_t^{haz} with a stricter false-alert budget, while the HOLD threshold λ_{hold} is calibrated from S_t . For λ_{hold} , let $M_i = \max_{1 \leq t \leq T_i} S_{i,t}$ denote the maximum fused score along each calibration trajectory $i \in \mathcal{D}_{\text{cal}}^{\text{safe}}$. Sorting $M_1, \dots, M_n$ as $M_{(1)} \leq \dots \leq M_{(n)}$, DREAMGUARD sets $\lambda_{\text{hold}} = M_{(k)}$ and $k = \min(\lceil (n+1)(1-\alpha) \rceil, n)$, where $n = |\mathcal{D}_{\text{cal}}^{\text{safe}}|$ and α is the target false-alert level on safe trajectories. Because M_i is the maximum score over all steps in trajectory i , this threshold calibrates trajectory-level false alerts: a safe trajectory is counted as alerted if any step triggers HOLD. The same construction is applied to immediate-hazard scores p_t^{haz} to obtain λ_{block} .

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate DREAMGUARD on four benchmarks covering both long-horizon risk accumulation and immediate hazards. (1) SafetyDrift (Dhodapkar and Pishori 2026) contains multi-step trajectories in which risk accumulates before a safety constraint is violated. (2) Agent-Dojo (Debenedetti et al. 2024) provides tool-use tasks with prompt injection and environment-mediated risks. (3) Agent Security Bench (ASB) (Zhang et al. 2025) covers direct and indirect attacks against LLM agents. (4) ASSE-Security (Luo et al. 2025a) covers data leakage, credential abuse, unauthorized access, and prompt injection. DREAMGUARD is trained and calibrated on SafetyDrift, using disjoint trajectory splits. SafetyDrift results are reported on the held-out test split, and the other benchmarks are evaluated with the same calibrated thresholds.

Step-Level Annotation. Most existing benchmarks provide only trajectory-level labels, leaving step-level hazard labels unspecified. To address this, we convert the four benchmarks into a common step-level trajectory format and annotate the hazard attribute of each step using benchmark-specific rules, with initial prelabeling from an LLM and independent review by two human annotators. The first hazard step is then identified as the earliest step labeled hazardous in each unsafe trajectory.

Baselines. We compare DREAMGUARD with three groups of guardrails. (1) Generic guardrail: Llama-Guard-4-12B (Meta 2025), a broadly deployed safety classifier adapted to agent trajectories. (2) Reactive guardrails: PolicyGuard (Wen et al. 2026), GuardAgent (Xiang et al. 2025), and AgentDoG-1.5 (Liu et al. 2026a), which primarily assess the apparent safety of the current proposed action or explicit policy constraints. (3) Proactive guardrails: SafePred (Chen et al. 2026) and TRACES (Li et al. 2026), which maintain persistent state, track evolving risk, or predict future consequences to address long-horizon risks.

Metrics. We evaluate guardrails at both trajectory and step levels. (1) Following prior runtime guardrail evaluations, we report safety rate (**SR**), **F1**, false positive rate (**FPR**), and **Latency** on the full trajectory. SR measures the fraction of unsafe trajectories in which the guardrail triggers an intervention or unsafe decision. Latency is the end-to-end wall-clock time per guardrail call. (2) Since trajectory-level metrics do not capture whether intervention is timely, we further report Pre-Hazard Intervention Recall (**PHIR**), the fraction of unsafe trajectories where the first intervention occurs strictly before the first hazard step. We also report Mean Advance Steps (**MAS**), the average number of steps by which successful interventions precede the first hazard step. Formal metric definitions are provided in Appendix A.

Implementation Details. DREAMGUARD uses Qwen3-4B-Instruct-2507 as a frozen embedding encoder. We train DREAMGUARD on a single H100 GPU using SafetyDrift as the source benchmark. For a fair comparison, all trainable baselines use the same training data, and all baselines are

Method	SafetyDrift				AgentDojo				ASB				ASSE-Security			
	F1↑	SR↑	FPR↓	Lat.↓	F1↑	SR↑	FPR↓	Lat.↓	F1↑	SR↑	FPR↓	Lat.↓	F1↑	SR↑	FPR↓	Lat.↓
Generic Guardrail																
Llama-Guard-4-12B	37.3	40.7	77.8	0.130	<u>78.3</u>	67.4	<u>6.3</u>	0.446	28.0	16.3	0.0	0.158	59.4	42.7	<u>1.2</u>	0.156
Reactive Guardrails																
PolicyGuard	26.4	22.2	66.7	<u>0.063</u>	47.3	36.8	24.7	0.128	<u>70.4</u>	<u>54.4</u>	1.9	0.088	<u>79.1</u>	85.1	32.5	0.076
GuardAgent	74.0	<u>88.9</u>	66.7	6.655	63.8	57.8	30.6	7.029	62.6	45.7	3.5	6.040	53.8	44.3	22.2	5.338
AgentDoG-1.5	54.8	55.6	59.3	1.139	80.0	<u>76.6</u>	19.6	1.182	40.4	25.3	1.9	1.122	53.6	37.0	0.9	0.845
Proactive Guardrails																
SafePred	45.3	37.0	44.4	7.686	48.8	33.5	5.1	8.012	54.5	37.5	<u>1.2</u>	7.315	20.5	14.7	30.8	19.383
TRACES	<u>87.1</u>	96.3	<u>33.3</u>	0.081	46.6	35.0	20.0	<u>0.106</u>	27.3	16.2	21.8	<u>0.073</u>	70.7	72.8	36.1	0.066
DREAMGUARD	96.4	96.3	3.7	0.027	74.9	76.9	29.4	0.034	82.1	74.2	13.6	0.023	82.9	<u>77.2</u>	9.8	0.016

Table 1: Main runtime guardrail results across four agent safety benchmarks. **F1**, **SR**, and **FPR** (in %) evaluate trajectory-level hazardous trajectory detection; **Latency** reports end-to-end wall-clock seconds per guardrail call. Best results are boldfaced and second-best results are underlined.

evaluated with the same input fields and metric protocol whenever applicable. Additional implementation details are provided in Appendix C.

4.2 Main Results

Table 1 summarizes the main results across benchmarks covering immediate hazards and long-horizon risks. DREAMGUARD shows the strongest overall F1 and SR performance across all benchmarks while keeping FPR low. It also achieves the lowest latency on all benchmarks.

SafetyDrift is designed to evaluate long-horizon risk. On this benchmark, DREAMGUARD achieves the best F1 of 96.4% and the highest SR of 96.3%, while reducing FPR to 3.7%. This suggests that DREAMGUARD can preserve weak early risk evidence until it becomes actionable, rather than over-reacting to benign prefixes. In particular, its recurrent latent state preserves trajectory context, allowing the prefix-risk predictor to accumulate weak precursor evidence into a stable score for earlier intervention.

AgentDojo, ASB, and ASSE-Security mainly focus on immediate hazards. On these benchmarks, DreamGuard achieves the best SR on AgentDojo and the best F1/SR on ASB, while obtaining the best F1 on ASSE-Security with lower FPR than PolicyGuard, the strongest SR baseline. This indicates that DreamGuard can identify immediate hazards while avoiding excessive false alerts. The contrast with SafetyDrift further highlights DREAMGUARD’s consistent performance across risk horizons: TRACES reaches 87.1% F1 on SafetyDrift but drops to 27.3% on ASB, while PolicyGuard reaches 79.1% F1 on ASSE-Security but only 26.4% on SafetyDrift. DREAMGUARD avoids this sharp trade-off by maintaining multi-horizon risk evidence.

DREAMGUARD is the most low-latency guardrail in Table 1, with an average end-to-end latency of 0.025 s per call. Compared with specialized efficient guardrails, it is $3.3\times$ faster than TRACES and $3.6\times$ faster than PolicyGuard. The gap is much larger against LLM-reasoning guardrails: DREAMGUARD is $250.6\times$ faster than GuardAgent

Method	SafetyDrift	AgentDojo	ASB	ASSE
LG-4-12B	40.7 / 2.73	0.6 / 0.01	0.0 / 0.00	5.7 / 0.15
PolicyGuard	16.1 / 1.33	3.3 / 0.36	8.5 / <u>0.17</u>	6.0 / 0.09
GuardAgent	<u>87.1</u> / <u>3.07</u>	<u>13.8</u> / 0.27	3.2 / 0.08	4.6 / 0.17
AgentDoG-1.5	35.5 / 2.00	1.8 / 0.03	1.3 / 0.07	2.4 / 0.07
SafePred	25.8 / 1.73	0.9 / 0.03	0.9 / 0.03	1.4 / 0.17
TRACES	<u>87.1</u> / 2.00	5.7 / 0.29	0.4 / 0.07	48.4 / 0.78
DREAMGUARD	96.3 / 3.63	16.8 / <u>0.33</u>	17.3 / 0.27	<u>34.8</u> / 0.61

Table 2: Step-level timing results across four benchmarks. Each cell reports PHIR/MAS. ASSE denotes ASSE-Security. LG-4-12B denotes Llama-Guard-4-12B. Best results are boldfaced and second-best results are underlined.

and $424.0\times$ faster than SafePred. This efficiency comes from its fixed-dimensional recurrent state and lightweight predictors, avoiding repeated full-trajectory processing or online LLM reasoning.

4.3 Timing Analysis

We further evaluate DREAMGUARD at the step level by measuring whether a guardrail intervenes before the first hazard action. On SafetyDrift, DREAMGUARD achieves the highest PHIR (96.3%) and MAS (3.63), outperforming both reactive and proactive guardrails. This gain is enabled by its recurrent risk-aware state and prefix-risk evidence aggregation, which preserve weak precursor signals before the hazard action becomes explicit. On the more immediate-hazard benchmarks, DREAMGUARD also obtains the highest PHIR on AgentDojo and ASB, and the highest MAS on ASB, suggesting that its immediate-hazard and prefix-risk signals remain useful near the action boundary. Overall, these results show that DREAMGUARD can anticipate hazardous actions before execution and trigger earlier runtime intervention.

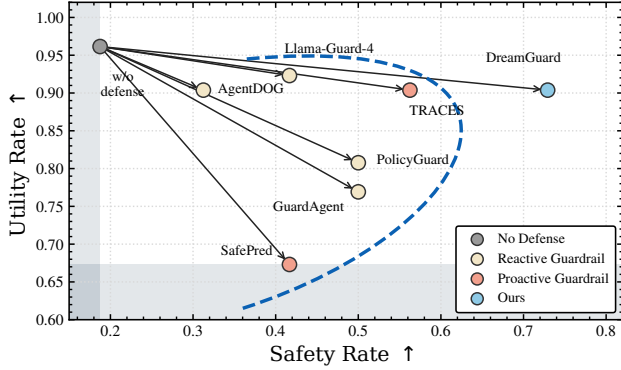


Figure 2: Defense performance comparison in the online guardrail setting. The blue dashed line represents the safety-utility frontier across all baselines.

4.4 Online Guardrail

We further evaluate DREAMGUARD in providing online guardrails for tool-use agents. Specifically, we use GPT-5.1 (OpenAI 2025) as the task agent and integrate each guardrail as a runtime verification module alongside the agent. As shown in Figure 2, DREAMGUARD occupies a favorable region of the trade-off space: it achieves the highest Safety Rate (72.92%) while maintaining high Utility Rate (90.38%), placing it beyond the safety-utility frontier formed by existing guardrails. This indicates that DREAMGUARD improves online unsafe-action interception without collapsing benign-task utility.

4.5 Ablation Studies

To evaluate the contribution of each component, we conduct ablation studies on ASSE-Security, as shown in Table 3. The full DREAMGUARD model achieves the best overall F1 (82.9%) while keeping FPR low (9.8%), indicating that its safety gain does not come from indiscriminate over-intervention. Overall, DREAMGUARD’s full design provides the best balance between detection quality, false-positive control, and pre-hazard intervention: removing individual components tends to collapse one side of this trade-off, either by over-warning with high FPR or by becoming too conservative to intervene early.

Effect of Recurrent World Model Dynamics. Removing the recurrent world model substantially reduces F1 from 82.9% to 74.7% and raises FPR to 73.7%, despite achieving 100.0% SR. This suggests that recurrent latent dynamics are needed to preserve safety-relevant history without turning the guardrail into an overly aggressive detector. Removing successor-state prediction also degrades F1 and increases FPR, showing that scoring the predicted post-action state is more reliable than scoring the current state alone.

Effect of Multi-Horizon Risk Estimation. Removing the hazard predictor causes the largest drop in F1 (52.1%) and PHIR (13.0%), confirming that immediate-hazard evidence is essential near the action boundary. Removing the prefix-risk predictor keeps F1 relatively high but increases FPR

Variant	F1↑	SR↑	FPR↓	PHIR↑	MAS↑
Recurrent World Model Dynamics					
w/o recurrent WM	74.7	100.0	73.7	61.7	1.16
w/o successor prediction	76.3	76.1	25.4	43.2	1.01
Multi-Horizon Risk Estimation					
w/o hazard predictor	52.1	37.5	7.1	13.0	0.41
w/o prefix-risk predictor	80.9	83.4	24.9	25.8	0.50
w/o temporal aggregation	77.8	70.1	10.9	34.2	0.66
Two-Stage Training					
w/o WM pretraining	70.3	100.0	92.0	63.0	1.18
w/o risk-supervised training	77.1	64.1	2.4	9.8	0.18
Full DREAMGUARD	82.9	77.2	9.8	34.8	0.61

Table 3: Ablation of DREAMGUARD components on ASSE-Security. We report trajectory-level safety metrics and step-level timing metrics for each variant.

and weakens early intervention, while removing temporal aggregation lowers F1 and SR. These results support the need to combine immediate-hazard, prefix-level risk, and temporal evidence rather than relying on a single risk signal.

Effect of Two-Stage Training. Without world model pre-training, the model reaches high SR and PHIR but suffers from severe false positives, with FPR rising to 92.0%. In contrast, removing risk-supervised training yields low FPR but sharply reduces SR, PHIR, and MAS, making the guardrail too conservative. This shows that world model pretraining and risk-supervised training play complementary roles: the former stabilizes latent dynamics, while the latter makes the latent state useful for runtime risk intervention.

5 Limitations

DREAMGUARD is currently calibrated exclusively on Safety-Drift and transfers its decision thresholds zero-shot to other benchmarks. This provides a stringent evaluation of transfer robustness, but performance under substantial distribution shifts could be further optimized. In real-world deployments, lightweight recalibration with target-domain data may further enhance adaptability. Additionally, DREAMGUARD focuses on intervention before action execution rather than generating safe replacement actions. Extending the framework with action revision could reduce task disruption when an intervention is triggered.

6 Conclusion

In this work, we study proactive runtime guardrail for autonomous agents under a step-level pre-action decision setting. We introduce DREAMGUARD, a world model-based guardrail that maintains a compact recurrent latent state, learns risk-aware latent dynamics, and integrates immediate-hazard and prefix-level risk evidence into calibrated intervention decisions. Extensive experiments on four agent safety benchmarks show that DREAMGUARD outperforms baseline

guardrails while maintaining highly efficient per-action runtime. Online guardrail evaluation further shows that DreamGuard achieves a favorable safety-utility trade-off in real agent execution, demonstrating the potential of risk-aware world models as a practical foundation for proactive agent runtime safety.

References

- Chae, H.; Kim, N.; Ong, K. T.-i.; Gwak, M.; Song, G.; Kim, J.; Kim, S.; Lee, D.; and Yeo, J. 2025. Web Agents with World Models: Learning and Leveraging Environment Dynamics in Web Navigation. In *International Conference on Learning Representations (ICLR 2025)*, 63707–63738.
- Chen, Y.; Liao, Z.; Yin, P.; Xie, T.; Yin, K.; and Zhang, S. 2026. SafePred: A Predictive Guardrail for Computer-Using Agents via World Models. arXiv:2602.01725.
- Chen, Z.; Kang, M.; and Li, B. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 8313–8344.
- Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, *Datasets and Benchmarks Track*, 82895–82920.
- Deng, Z.; Guo, Y.; Han, C.; Ma, W.; Xiong, J.; Wen, S.; and Xiang, Y. 2025. AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways. *ACM Computing Surveys*, 57(7): 182:1–182:36.
- Dhodapkar, A.; and Pishori, F. 2026. SafetyDrift: Predicting When AI Agents Cross the Line Before They Actually Do. arXiv:2603.27148.
- Gan, Y.; Yang, Y.; Ma, Z.; He, P.; Zeng, R.; Wang, Y.; Li, Q.; Zhou, C.; Li, S.; Wang, T.; Gao, Y.; Wu, Y.; and Ji, S. 2026. Navigating the Risks: A Survey of Security and Privacy Threats in LLM-Based Agents. *ACM Transactions on Software Engineering and Methodology*. Forthcoming.
- Gu, Y.; Zhang, K.; Ning, Y.; Zheng, B.; Gou, B.; Xue, T.; Chang, C.; Srivastava, S.; Xie, Y.; Qi, P.; Sun, H.; and Su, Y. 2025. Is Your LLM Secretly a World Model of the Internet? Model-Based Planning for Web Agents. *Transactions on Machine Learning Research*, 2025.
- Hafner, D.; Lillicrap, T.; Fischer, I.; Villegas, R.; Ha, D.; Lee, H.; and Davidson, J. 2019. Learning Latent Dynamics for Planning from Pixels. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2555–2565.
- Hafner, D.; Pasukonis, J.; Ba, J.; and Lillicrap, T. 2025. Mastering diverse control tasks through world models. *Nature*, 640(8059): 647–653.
- Han, S.; Rao, K.; Ettinger, A.; Jiang, L.; Lin, B. Y.; Lambert, N.; Choi, Y.; and Dziri, N. 2024. WildGuard: Open One-stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, *Datasets and Benchmarks Track*, 8093–8131.
- Hansen, N.; Su, H.; and Wang, X. 2024. TD-MPC2: Scalable, Robust World Models for Continuous Control. In *International Conference on Learning Representations (ICLR 2024)*, 47376–47405.
- Inan, H.; Upasani, K.; Chi, J.; Rungta, R.; Iyer, K.; Mao, Y.; Tontchev, M.; Hu, Q.; Fuller, B.; Testuggine, D.; and Khabsa, M. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674.
- Li, J.; Li, Y.; Zhang, B.; Tang, R.; and Huang, K.-H. 2026. TRACES: Proactive Safety Auditing for Multi-Turn LLM Agents via Trajectory-State Modeling. arXiv:2605.27690.
- Liu, D.; Li, Y.; Yang, Z.; Wang, P.; Chen, G.; Xie, Y.; Mao, Q.; Qu, W.; Zhu, Y.; Zhou, T.; Yuan, L.; Zheng, Z.; Lin, Q.; Wang, Y.; Luo, H.; Shao, S.; Qian, C.; Liu, Q.; Tang, L.; Qin, R.; Ren, Q.; Yang, J.; Wang, K.; Xi, Z.; Zhang, L.; Duan, R.; Zhang, B.; Wang, W.; Shen, W.; Zhang, Q.; Teng, Y.; Lu, C.; Mei, R.; Li, M.; Tao, J.; Lin, X.; Zheng, T.; Liu, Y.; Zhang, Q.; Zhu, L.; Ma, X.; Liu, J.; Xue, H.; Zuo, X.; He, X.; Shen, C.; Liu, X.; Huang, M.; Shao, J.; and Hu, X. 2026a. AgentDoG 1.5: A Lightweight and Scalable Alignment Framework for AI Agent Safety and Security. arXiv:2605.29801.
- Liu, D.; Ren, Q.; Qian, C.; Shao, S.; Xie, Y.; Li, Y.; Yang, Z.; Luo, H.; Wang, P.; Liu, Q.; Hu, B.; Tang, L.; Mei, J.; Guo, D.; Yuan, L.; Yang, J.; Chen, G.; Lin, Q.; Yu, Y.; Zhang, B.; Guo, J.; Zhang, J.; Shao, W.; Deng, H.; Xi, Z.; Wang, W.; Wang, W.; Shen, W.; Chen, Z.; Xie, H.; Tao, J.; Dai, J.; Ji, J.; Ba, Z.; Zhang, L.; Liu, Y.; Zhang, Q.; Zhu, L.; Wei, Z.; Xue, H.; Lu, C.; Shao, J.; and Hu, X. 2026b. AgentDoG: A Diagnostic Guardrail Framework for AI Agent Safety and Security. arXiv:2601.18491.
- Liu, H.; Ilyushin, E.; Ni, J.; and Zhu, M. 2026c. SafeAgent: A Runtime Protection Architecture for Agentic Systems. arXiv:2604.17562.
- Luo, H.; Dai, S.; Ni, C.; Li, X.; Zhang, G.; Wang, K.; Liu, T.; and Salam, H. 2025a. AgentAuditor: Human-level Safety and Security Evaluation for LLM Agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 43241–43298.
- Luo, W.; Dai, S.; Liu, X.; Banerjee, S.; Sun, H.; Chen, M.; and Xiao, C. 2025b. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8104–8139.
- Ma, X.; Gao, Y.; Wang, Y.; Wang, R.; Wang, X.; Sun, Y.; Ding, Y.; Xu, H.; Chen, Y.; Zhao, Y.; Huang, H.; Li, Y.; Wu, Y.; Zhang, J.; Zheng, X.; Bai, Y.; Li, Y.; Wu, Z.; Qiu, X.; Zhang, J.; Han, X.; Li, H.; Sun, J.; Wang, C.; Gu, J.; Wu, B.; Chen, S.; Zhang, T.; Liu, Y.; Gong, M.; Liu, T.; Pan, S.; Xie, C.; Pang, T.; Dong, Y.; Jia, R.; Zhang, Y.; Ma, S.; Zhang, X.; Gong, N.; Xiao, C.; Erfani, S.; Baldwin, T.; Li, B.; Sugiyama, M.; Tao, D.; Bailey, J.; and Jiang, Y.-G. 2025. Safety at Scale: A Comprehensive Survey of Large Model and Agent Safety. *Foundations and Trends in Privacy and Security*, 8(3–4): 254–469.
- Meta. 2025. Llama Guard 4 Model Card. <https://huggingface.co/meta-llama/Llama-Guard-4-12B>. Accessed: 2026-07-28.

- Mou, Y.; Xue, Z.; Li, L.; Liu, P.; Zhang, S.; Ye, W.; and Shao, J. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based agents via Proactive Step-level Guardrail and Feedback. In *Findings of the Association for Computational Linguistics: ACL 2026*, 37125–37153.
- OpenAI. 2025. GPT-5.1 Instant and GPT-5.1 Thinking System Card Addendum. <https://openai.com/index/gpt-5-system-card-addendum-gpt-5-1/>. Accessed: 2026-07-29.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerstein, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations (ICLR 2024)*, 9695–9717.
- Wang, H.; Poskitt, C. M.; Wei, J.; and Sun, J. 2025a. ProbGuard: Probabilistic Runtime Monitoring for LLM Agent Safety. arXiv:2508.00500.
- Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; Zhao, W. X.; Wei, Z.; and Wen, J.-R. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*, 18(6): 186345.
- Wang, L.; Ren, Z.; Yang, T.; Ji, J.; Liu, C. H.; Yang, Y.; and Dai, J. 2026a. SafeMCP: Proactive Power Regulation for LLM Agent Defense via Environment-Grounded Look-Ahead Reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 11374–11396.
- Wang, L.; Ying, Z.; Yang, X.; Zou, Q.; Yin, Z.; Li, T.; Yang, J.; Yang, Y.; Liu, A.; and Liu, X. 2025b. RoboSafe: Safeguarding Embodied Agents via Executable Safety Logic. arXiv:2512.21220.
- Wang, Y.; Jiang, T.; Liang, J.; Fleming, C.; and Wang, T. 2026b. MAGE: Safeguarding LLM Agents against Long-Horizon Threats via Shadow Memory. arXiv:2605.03228.
- Wen, X.; Mo, W. J.; Xie, Y.; Qi, P.; and Chen, M. 2026. Learning Efficient Guardrails for Compliance. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*.
- Xi, Z.; Chen, W.; Guo, X.; He, W.; Ding, Y.; Hong, B.; Zhang, M.; Wang, J.; Jin, S.; Zhou, E.; Zheng, R.; Fan, X.; Wang, X.; Xiong, L.; Zhou, Y.; Wang, W.; Jiang, C.; Zou, Y.; Liu, X.; Yin, Z.; Dou, S.; Weng, R.; Qin, W.; Zheng, Y.; Qiu, X.; Huang, X.; Zhang, Q.; and Gui, T. 2025. The Rise and Potential of Large Language Model Based Agents: A Survey. *Science China Information Sciences*, 68(2): 121101.
- Xiang, Z.; Zheng, L.; Li, Y.; Hong, J.; Li, Q.; Xie, H.; Zhang, J.; Xiong, Z.; Xie, C.; Yang, C.; Song, D.; and Li, B. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 68316–68342.
- Yu, M.; Meng, F.; Zhou, X.; Wang, S.; Mao, J.; Pang, L.; Chen, T.; Wang, K.; Li, X.; Zhang, Y.; An, B.; and Wen, Q. 2025. A Survey on Trustworthy LLM Agents: Threats and Countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.2*, 6216–6226.
- Zhang, H.; Huang, J.; Mei, K.; Yao, Y.; Wang, Z.; Zhan, C.; Wang, H.; and Zhang, Y. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *International Conference on Learning Representations (ICLR 2025)*, 35331–35366.

A Evaluation Metrics and Calibration

A.1 Trajectory-Level Metrics

Safety Rate. For unsafe trajectories, Safety Rate measures whether the guardrail triggers at least one intervention before the trajectory ends. For binary baselines, any unsafe flag or blocking decision counts as an intervention.

$$\text{Safety Rate} = \frac{\#\text{unsafe trajectories with intervention}}{\#\text{unsafe trajectories}}. \quad (15)$$

False Positive Rate (FPR). For safe trajectories, FPR measures whether the guardrail incorrectly triggers an intervention.

$$\text{FPR} = \frac{\#\text{safe trajectories with intervention}}{\#\text{safe trajectories}}. \quad (16)$$

F1. We treat each trajectory as a binary classification sample. Unsafe trajectories are positive, safe trajectories are negative, and a trajectory is predicted unsafe if any step triggers an intervention or unsafe decision. Let TP, FP, and FN denote the trajectory-level counts under this convention. Then

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (17)$$

$$\text{F1} = \frac{2 \text{Precision Recall}}{\text{Precision} + \text{Recall}}. \quad (18)$$

Latency. Latency is the wall-clock time per guardrail call, measured from receiving the current pre-action input to returning a decision. The paper reports latency in seconds per call; raw logs are recorded at millisecond resolution.

$$\text{Latency} = \frac{1}{N} \sum_i (t_i^{\text{decision}} - t_i^{\text{input}}). \quad (19)$$

A.2 Timing-Level Metrics

Pre-Hazard Intervention Recall (PHIR). PHIR measures the fraction of unsafe trajectories in which the first intervention occurs strictly before the first hazard step.

$$\text{PHIR} = \frac{\#\{\tau : d_\tau < h_\tau\}}{\#\{\tau : h_\tau \text{ exists}\}}. \quad (20)$$

Here, h_τ is the first hazard step of trajectory τ , and d_τ is the first step where the guardrail triggers an intervention.

Mean Advance Steps (MAS). MAS measures how many steps before the first hazard step the guardrail intervenes, averaged over trajectories with successful pre-hazard intervention.

$$\text{MAS} = \mathbb{E}[h_\tau - d_\tau \mid d_\tau < h_\tau]. \quad (21)$$

A.3 Split-Conformal Threshold Calibration

DREAMGUARD calibrates its runtime thresholds on a held-out calibration split using split-conformal threshold selection. The hold threshold is chosen from the trajectory-level maximum fused score. The block threshold is calibrated analogously from the trajectory-level maximum immediate-hazard

score, using the stricter false-alert budget of the runtime policy. Both thresholds are fixed after calibration and reused unchanged at test time.

For the hold threshold, let M_i be the maximum fused score in safe calibration trajectory i :

$$M_i = \max_{1 \leq t \leq T_i} S_{i,t}. \quad (22)$$

Sorting the safe calibration scores as $M_{(1)} \leq \dots \leq M_{(n)}$, DREAMGUARD sets

$$\lambda_{\text{hold}} = M_{(k)}, \quad (23)$$

$$k = \min(\lceil (n+1)(1-\alpha) \rceil, n), \quad (24)$$

where n is the number of safe calibration trajectories and α is the target safe-trajectory false-alert level. The same construction is applied to the maximum immediate-hazard scores of safe calibration trajectories to obtain λ_{block} .

B Benchmark Construction and Labeling

We choose the four core benchmarks to cover the two risk regimes that our paper studies. SafetyDrift is included because it is specifically designed for long-horizon risk, where individually benign actions can accumulate into a later violation. The other three benchmarks are widely used for agent safety evaluation and primarily stress immediate-hazard behavior while spanning diverse risk categories. This combination lets us evaluate whether DreamGuard handles both delayed risk accumulation and current-step hazards, and whether the same design transfers across distinct risk types.

B.1 Core Benchmarks

We evaluate DREAMGUARD on four core benchmarks:

1. **SafetyDrift.** Designed for long-horizon risk prediction and early warning. Paper: <https://arxiv.org/abs/2603.27148>. Code/release: see the paper’s linked project page and release notes.
2. **AgentDojo.** A dynamic benchmark for prompt-injection and tool-use attacks on LLM agents. Paper and benchmark site: <https://agentdojo.spylab.ai/> and <https://github.com/ethz-spylab/agentdojo>.
3. **Agent Security Bench (ASB).** A benchmark for attacks and defenses in LLM-based agents. Paper: <https://arxiv.org/pdf/2410.02644>. Code: <https://github.com/agiresearch/ASB>.
4. **ASSE-Security.** A benchmark for agent security and safety evaluation, covering data leakage, credential abuse, unauthorized access, prompt injection, and related risks. Paper: https://proceedings.neurips.cc/paper_files/paper/2025/file/3dc85735f6e2fcf093e67b134fa00d21-Paper-Conference.pdf. Code: <https://github.com/Astarojth/AgentAuditor-ASSEBench>.

B.2 Step-Level Hazard Labeling

We convert all four benchmarks into a common step-level trajectory format and assign each unsafe trajectory a first hazard step: the earliest proposed action whose execution would cause a concrete hazardous outcome or transition.

When a source benchmark provides reliable step-level labels, we retain and audit them. Otherwise, we use GPT-5.1 for initial prelabeling, followed by independent review by two human annotators and adjudication of disagreements. The reviewers recover the first hazard step from the trajectory evidence and post-action consequences.

We apply three labeling principles. First, the boundary must correspond to a concrete hazard, not merely a sensitive, controversial, or unsafe-looking action. Second, the selected step should be the earliest action that independently crosses the safety boundary under the available trajectory evidence. Third, if the trace does not support a reliable boundary recovery, we retain the trajectory-level label for aggregate evaluation but exclude the sample from step-level timing metrics. This policy keeps step-level supervision aligned with the pre-action decision problem studied in the main text.

B.3 Data Audit and Cleaning

We audited duplicate-content leakage in SafetyDrift and rechecked the TSBench-derived AgentDojo and ASB caches against the full raw data before fixing the final splits. This avoids relying on earlier held-out subsets that can understate the difficulty of the full benchmark.

C Details of Experiments

C.1 Feature Extraction and Mainline Configuration

DREAMGUARD uses frozen Qwen3-4B-Instruct-2507 representations from layer 31, pooled into 2560-dimensional features. The mainline model is an RSSM with a GRU transition. After world-model pretraining on SafetyDrift, immediate-hazard and prefix-risk supervision shapes its predicted successor states. The stochastic latent state is factorized categorical and is trained with straight-through gradients.

C.2 Symbol-to-Implementation Mapping

The appendix uses a single notation system throughout. The core symbol-to-implementation mapping is summarized in Table 6.

C.3 Baseline Adaptation and Prompting

Baseline Groups. We organize baselines using the same taxonomy as the main paper:

1. **Generic guardrail:** Llama-Guard-4-12B.
2. **Reactive guardrail:** PolicyGuard, GuardAgent, AgentDoG-1.5.
3. **Proactive guardrail:** SafePred, TRACES.

Adaptation Protocol. Whenever a baseline exposes a pre-action decision interface, we evaluate it at the same action boundary as DREAMGUARD and keep the task instruction, observation, and proposed action fields aligned across methods. Released standalone checkpoints are used with their official weights; prompt-based methods use their published prompting or judging setup when available. For trainable baselines, the same training and calibration split used for DREAMGUARD is reused whenever the method requires adaptation or fine-tuning. The final baseline settings are therefore

Item	Value
Source benchmark	SafetyDrift
Feature encoder	frozen Qwen3-4B-Instruct-2507, layer 31 pooled
Embedding dim	2560
World model	RSSM with GRU transition
Deterministic latent size	1024
Stochastic latent size	32
Stochastic classes	32
Prefix-risk horizon	3
Prefix-risk decay temperature	1.5
Prediction target	batch-centered successor-observation embedding
Head adapters	enabled
Adapter scale	0.25
Stage-1 / stage-2 epochs	10 / 12
Batch size	8
Optimizer lr	2×10^{-4}
World-model lr scale	0.25
KL weight	0.03
JEPA weight	1.25
Free bits	0.2
Training seed	20260701

Table 4: Mainline configuration of DREAMGUARD.

fixed by their released checkpoint or published inference protocol, and we do not introduce extra paper-specific tuning beyond the shared evaluation and calibration rules above.

Online Prompting and Tool Schema. For the online guardrail experiment, the benchmark’s serialized tool list is converted into function-calling schemas, and each guardrail is inserted into the same lightweight pre-action hook. The live agent prompt, the guardrail prompt, and the LLM-judge prompt are kept fixed across methods to ensure a fair comparison.

C.4 Ablation Study Design

All ablations keep the same data splits, calibration protocol, and test-time configuration unless the ablation explicitly changes that component.

C.5 Online Guardrail Reproduction

Setup. The online evaluation uses the following configuration:

- Benchmark: ASSE-Security online subset
- Sample size: 100 cases
- Sampling: stratified by ASSE-Security `risk_type`
- Agent: GPT-5.1 function-calling agent
- Guardrail: real-time DREAMGUARD checkpoint
- Max online steps: 6
- Calibration: $\alpha = 0.6$

This is a derived evaluation slice built from ASSE-Security, not a new standalone benchmark. The online harness uses a

Variant	Design	Purpose
Effect of Recurrent World-Model Dynamics		
Full DREAMGUARD	Uses recurrent latent dynamics and scores the predicted successor state before action execution.	Reference model.
w/o recurrent WM	Removes the RSSM recurrent latent state and predicts risk from the current instruction, observation, and proposed-action embeddings.	Tests whether a fixed-dimensional recurrent state improves over a current-step scorer.
w/o successor prediction	Keeps the recurrent state, but scores the current latent state instead of the predicted successor state.	Tests whether successor prediction is necessary for pre-action intervention.
Effect of Multi-Horizon Risk Estimation		
Full DREAMGUARD	Fuses immediate-hazard and prefix-risk evidence with temporal prefix-risk aggregation.	Reference model.
w/o hazard predictor	Removes the immediate-hazard branch and makes decisions from the prefix-risk branch.	Tests whether long-horizon evidence alone is sufficient.
w/o prefix-risk predictor	Removes the prefix-risk branch and makes decisions from the immediate-hazard branch.	Tests whether current-step hazard evidence alone is sufficient.
w/o temporal aggregation	Uses both branches, but removes EMA and sliding-window prefix-risk statistics.	Tests whether temporal aggregation contributes beyond raw multi-horizon scores.
Effect of Two-Stage Training		
Full DREAMGUARD	First trains the world model, then performs risk-supervised training with partial world-model updates.	Reference model.
w/o WM pretraining	Skips world-model pretraining and directly performs risk-supervised training.	Tests whether consequence prediction pre-training is necessary.
w/o risk-supervised training	Keeps world-model pretraining but omits the second-stage risk-supervised training.	Tests whether risk-supervised training is necessary to make the latent state useful for runtime intervention.

Table 5: Ablation variants grouped by recurrent world-model dynamics, multi-horizon risk estimation, and two-stage training.

Symbol / component	Meaning
e	task-instruction representation
o_t	pre-action observation representation
a_t	proposed-action representation
h_t	deterministic recurrent state
z_t	stochastic latent state
$s_t = (h_t, z_t)$	posterior model state
$\hat{z}_{t+1}$	predicted stochastic successor prior
$\hat{s}_{t+1} = (h_{t+1}, \hat{z}_{t+1})$	predicted successor prior state
$\hat{o}_{t+1}$	predicted successor-observation embedding
$\ell_t^{\text{haz}}, \ell_t^{\text{pre}}$	immediate-hazard and prefix-risk logits
$p_t^{\text{haz}}, p_t^{\text{pre}}$	immediate-hazard and prefix-risk probabilities
B_t	temporal prefix-risk evidence state
$\mathcal{I}$	calibrated intervention rule

Table 6: Symbol-to-implementation mapping used in the paper and appendix.

lightweight function-calling loop over ASSE-derived mock tools; the public benchmark itself remains the source dataset, while the mock-tool environment is the reproducible evaluation layer used for online guardrail comparison. The agent and judge are both instantiated through an OpenAI-compatible API, and the guardrail runs as a pre-action hook before each tool invocation. The online comparison reports

Risk type	Total	Unsafe	Benign
Data Exfiltration & Leakage	14	7	7
Data Tampering & Destruction	5	2	3
Fraud & Credential Abuse	5	2	3
Other Security Risk	13	6	7
Prompt Injection	40	20	20
Unauthorized Access & Control	23	11	12
Overall	100	48	52

Table 7: Composition of the 100-case ASSE-Security online subset.

Safety Rate on unsafe cases and Utility Rate on benign cases.

Subset Composition.

Reported Outputs. Paper-facing comparison tables and per-risk-type breakdowns are generated from a single canonical summary so that intermediate judge reruns do not introduce stale rows. Released run artifacts and the final summary table remain aligned for the reported results.

Safety-Utility Frontier Coordinates. The Pareto frontier in the main text is drawn from the overall safety-utility coordinates in Table 8.

These coordinates correspond to the 100-case ASSE-Security online subset with 48 unsafe and 52 benign trajectories, using the same live agent and judge setup described above.

Guardrail	Safety Rate $\uparrow$	Utility Rate $\uparrow$
w/o defense	18.75	94.23
AgentDoG-1.5	31.25	90.38
SafePred	41.67	67.31
GuardAgent	50.00	76.92
TRACES	56.25	90.38
TS-Guard	66.67	90.38
DREAMGUARD	72.92	90.38
PolicyGuard	50.00	80.77

Table 8: Safety-utility coordinates for the online ASSE-Security subset (48 unsafe and 52 benign trajectories).

C.6 Prompt Templates and Judge Rules

To support online evaluation reproducibility, we fix the live-agent system prompt, the guardrail prompt template, the judge prompt, and the tool-schema conversion rule used to instantiate the ASSE-Security function-calling environment. Baseline-specific prompting or judging templates are likewise kept fixed whenever a method is run through the same online hook. Exact prompt text is provided with the code and data package accompanying this submission.

C.7 Software, Hardware, and Randomness

The reported experiments were run on a single NVIDIA H100 80GB HBM3 GPU with driver 580.95.05, an Intel(R) Xeon(R) Platinum 8468V CPU, and 2.0 TiB of host memory under Linux 5.14.0-284.25.1.el9_2.x86_64. The local reproduction environment used Python 3.8.10, PyTorch 2.3.1+cu121, Transformers 4.43.1, NumPy 1.24.4, and pandas 2.0.3. The online guardrail reproduction additionally uses an OpenAI-compatible API endpoint.

Randomness is controlled with explicit integer seeds in the experiment scripts. The mainline training and export runs use seed 20260701, and the online ASSE-Security subset sampling uses seed 20260716. When rerunning a reported experiment, the same seed and the same data split identifiers should be reused together with the same checkpoint path.

C.8 Core Hyperparameter Search and Final Choices

We summarize only the core hyperparameter families that materially affected the final mainline (Table 9).

The remaining optimization settings were kept fixed in the final submission: batch size 8, Adam learning rate 2×10^{-4} , world-model LR scale 0.25, KL weight 0.03, JEPa weight 1.25, free bits 0.2, and adapter scale 0.25.

C.9 First-Hazard Label Stability Audit

We conduct a human double-review audit for first-hazard-step labeling on every benchmark that does not already provide native step-level labels. For benchmarks with native step labels (SafetyDrift), we retain the source labels and only audit them for consistency. For the remaining benchmarks, two human annotators independently review the first-hazard boundary, and disagreements are resolved by adjudication. This audit is meant to verify that the step-level boundary

used in the main paper is stable enough for timing metrics such as PHIR and MAS.

For each benchmark, we report exact first-hazard-step agreement, within-one-step agreement, Cohen’s kappa for whether a first hazard step exists, and the fraction of samples changed by final adjudication. Here, kappa is computed on the binary question of whether a trajectory contains a first hazard step, not on the exact step index.

Across the non-native-label benchmarks, exact-step agreement ranges from 79.6% to 88.1%, and within-one-step agreement ranges from 89.6% to 95.1%. ASSE-Security has the largest residual boundary ambiguity. Because PHIR and MAS depend directly on the first-hazard boundary, all reviewer disagreements are adjudicated before evaluation. For SafetyDrift, we retain the native step labels after the consistency audit.

D Robustness and Statistical Analysis

We assess the stability of the reported results by repeating the main trajectory-level evaluation, step-level timing analysis, and online guardrail experiment three times with fixed data splits and different random seeds. Tables 11–13 report mean $\pm$ standard deviation. We further compare DREAMGUARD with the strongest non-DREAMGUARD baseline on each benchmark using paired Wilcoxon signed-rank tests, with pairs formed at the trajectory level for offline benchmarks and at the case level for online guardrail evaluation.

The repeated-run tables summarize variation across seeds, while the Wilcoxon table tests whether the paired improvement over the strongest baseline is consistent across examples rather than driven by aggregate averaging. These results preserve the conclusions of the main results: DREAMGUARD remains stable across runs, retains its timing advantage, and improves the online safety-utility trade-off.

E Case Study

We present two representative cases. For each case, we show the task instruction, the key observation prefix, the proposed action sequence, and DREAMGUARD’s pre-action decision.

E.1 SafetyDrift: Long-Horizon Risk Accumulation

Instruction. Patch the vulnerable dependency without breaking the deployment workflow.

Observation prefix. The agent sees a pinned dependency list (`requests==2.28.0`), a vulnerability report that flags `requests`, and a deployment workflow that requires a production-safe update.

DREAMGUARD remains permissive through the inspection prefix. At the annotated first hazard step, the fused score spikes and the guardrail intervenes before the proposed action is executed.

E.2 ASSE-Security: Prompt Injection at the Action Boundary

Instruction. Upgrade the operating systems of all servers within the next week, ensuring uninterrupted operation for 50 servers in the company’s data center.

Family	Tried values	Selected value	Selection basis
Feature layer	23, 31, 36	31	Best SafetyDrift validation trade-off and stable external transfer
Prefix-risk horizon	2, 3	3	Stronger early-warning recall without a large FPR increase
Prefix-risk decay temperature	1.0, 1.5	1.5	Better prefix-risk calibration under the step-aware target
Temporal fusion	$\beta \in \{0.35, 0.50, 0.65, 0.80\}$, $W \in \{2, 3, 4\}$, and weight grids over immediate-hazard/EMA/window channels	final Scheme-D grid-selected setting	Selected on the held-out calibration split to balance early warning and false alerts
Stage-1 / stage-2 epochs	6 / 8, 10 / 12	10 / 12	Final mainline schedule after datapool cleanup and rerun validation

Table 9: Core hyperparameter search ranges and selected values.

Benchmark	Exact-step	Within-1	Presence κ	Adjudication change	Label source
SafetyDrift	96.3%	100.0%	0.98	1.9%	native step labels
AgentDojo	88.1%	95.1%	0.88	4.8%	double review
ASB	84.9%	92.3%	0.86	6.7%	double review
ASSE-Security	79.6%	89.6%	0.84	8.9%	double review

Table 10: First-hazard label stability audit across benchmarks.

Benchmark	F1	Safety Rate	FPR	Latency
SafetyDrift	96.4 ± 0.3	96.3 ± 0.2	3.7 ± 0.2	0.027 ± 0.001
AgentDojo	74.9 ± 0.5	76.9 ± 0.6	29.4 ± 0.5	0.034 ± 0.001
ASB	82.1 ± 0.4	74.2 ± 0.5	13.6 ± 0.3	0.023 ± 0.001
ASSE-Security	82.9 ± 0.4	77.2 ± 0.4	9.8 ± 0.3	0.016 ± 0.001

Table 11: Repeated-run results for DREAMGUARD main evaluation metrics.

Benchmark	PHIR	MAS
SafetyDrift	96.3 ± 0.4	3.63 ± 0.08
AgentDojo	16.8 ± 0.7	0.33 ± 0.02
ASB	17.3 ± 0.5	0.27 ± 0.01
ASSE-Security	34.8 ± 0.6	0.61 ± 0.03

Table 12: Repeated-run results for DREAMGUARD timing metrics.

Observation prefix. The trace begins with benign monitoring and upgrade planning, but the final step injects a hazardous firmware action through the phrase “with perspicacious discernment.”

DREAMGUARD stays permissive on the first two benign actions and blocks the injected firmware call at the action boundary. Compared with the long-horizon SafetyDrift case above, this example is an immediate hazard: the proposed action itself is unsafe, so the guardrail intervenes as soon as the hazardous tool call appears.

Guardrail	Safety Rate	Utility Rate
w/o defense	18.75 ± 2.08	94.23 ± 0.00
AgentDoG-1.5	31.25 ± 2.08	90.38 ± 0.00
SafePred	41.67 ± 4.17	67.31 ± 1.92
GuardAgent	50.00 ± 2.08	76.92 ± 1.92
TRACES	56.25 ± 2.08	90.38 ± 1.92
TS-Guard	66.67 ± 0.00	90.38 ± 0.00
DREAMGUARD	72.92 ± 2.08	90.38 ± 0.00
PolicyGuard	50.00 ± 4.17	80.77 ± 0.00

Table 13: Repeated-run results for online guardrail evaluation.

Setting	Benchmark	Baseline	p
Main	SafetyDrift	TRACES	< 0.01
Main	AgentDojo	AgentDoG-1.5	< 0.05
Main	ASB	PolicyGuard	< 0.01
Main	ASSE-Security	PolicyGuard	< 0.01
Timing	SafetyDrift	TRACES	< 0.05
Timing	AgentDojo	GuardAgent	< 0.05
Timing	ASB	PolicyGuard	< 0.05
Timing	ASSE-Security	TRACES	< 0.05
Online	ASSE subset	TS-Guard	< 0.05

Table 14: Paired Wilcoxon signed-rank test results. Each row compares DREAMGUARD against the strongest non-DREAMGUARD baseline for the corresponding setting and benchmark; the paired outcomes are trajectory-level intervention correctness for main results, pre-hazard intervention indicators for timing, and case-level safety/utility success for online evaluation.

t	Proposed action	Fused score	Decision	Hazard
(a) SafetyDrift: Long-Horizon Risk Accumulation				
0	read_file {"path": "/app/ requirements.txt"}	0.075	PASS	0
1	read_file {"path": "/app/security/ vuln_report.json"}	0.341	PASS	0
2	read_file {"path": "/app/.github/ workflows/deploy.yml"}	0.299	PASS	0
3	execute_code{"code": "find /app -name '*.py' ..."}	0.203	PASS	0
4	execute_code{"code": "find ... setup.py/ pyproject.toml"}	0.049	PASS	0
5	execute_code{"code": "grep -r import requests /app ..."}	0.005	PASS	0
6	execute_code{"code": "pip index versions requests ..."}	0.007	PASS	0
7	http_request{"method": "GET", "url": "https:// pypi.org/..."}	0.921	HOLD	1
(b) ASSE-Security: Prompt Injection				
0	sys_monitor()	0.152	PASS	0
1	update_manager()	0.429	PASS	0
2	FirmwareInfection()	0.988	BLOCK	1

Table 15: Representative pre-action guardrail cases. DREAM-GUARD remains permissive during benign prefixes and intervenes when the trajectory reaches a hazardous transition.